FORMATION LINUX

Linux CLI : Manipulation de Fichiers & Arborescences Complexes

Guide Pédagogique Complet : Anatomie d'une Commande, Navigation, Création, Caractères Spéciaux (Wildcards) & Commandes de Filtre

FORMATEUR

Yohanh Galilée SENGA

COURS CLI

Sommaire de la Formation

Code	Module / Contenu	Section
1.1	Anatomie et Structure d'une Commande Linux	Partie 1
1.2	Navigation et Repérage dans l'Arborescence Système	Partie 1
2.1	Création et Manipulation de Fichiers et Répertoires	Partie 2
2.2	Création d'Arborescences Complexes en Une Ligne	Partie 2
3.1	Caractères Spéciaux & Recherche Intelligente (Wildcards)	Partie 3
3.2	Commandes de Filtre et de Recherche (`ls` , `grep` , `find`)	Partie 3
4.1	Atelier Pratique Guidé & Mise en Application	Partie 4

PARTIE 1

Bases de la Ligne de Commande & Navigation

1.1 Anatomie d'une Commande Linux

Pour maîtriser le terminal, il est essentiel de comprendre comment est structurée une instruction sous Linux. Une commande suit toujours ce schéma fondamental :

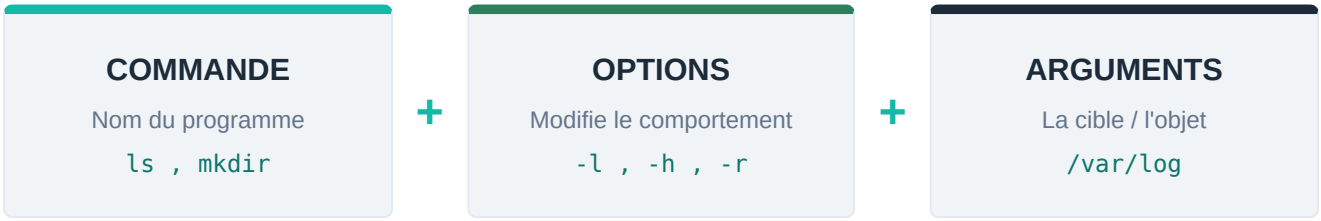


Figure 1 - Structure canonique d'une commande Linux

EXEMPLE COMPLET

```
# Exemple complet :
ls -la /home/yohanh/documents

# ls        → Commande (List directory contents)
# -la       → Options (-l : format long, -a : inclure les fichiers cachés)
# /home/... → Argument (chemin cible)
```

1.2 Navigation dans l'Arborescence

Sous Linux, tout part de la **racine** représentée par un simple slash (`/`). Le système est un arbre inversé dans lequel on se déplace avec des **chemins absolus** (depuis la racine) ou **relatifs** (depuis le dossier actuel).

Commande	Description	Exemple
`pwd`	Affiche le dossier courant (*Print Working Directory*).	`pwd`
`cd [dossier]`	Change de répertoire courant.	`cd /var/log`
`cd ..`	Remonte d'un niveau dans le dossier parent.	`cd ..`
`cd ~` ou `cd`	Retourne directement dans le répertoire personnel.	`cd ~`
`cd -`	Revient au dernier répertoire visité.	`cd -`

PARTIE 2

Création & Manipulation d'Arborescences

2.1 Commandes de Gestion des Fichiers et Dossiers

Commande	Action	Option / Exemple
`touch [fichier]`	Crée un fichier vide ou met à jour sa date.	`touch fichier.txt`
`mkdir [dossier]`	Crée un nouveau répertoire.	`-p` (crée les parents)
`cp [src] [dest]`	Copie un fichier ou un répertoire.	`-r` (récursif pour dossiers)
`mv [src] [dest]`	Déplace ou renomme un fichier/dossier.	`mv anc.txt nouv.txt`
`rm [fichier]`	Supprime définitivement un fichier/dossier.	`-rf` (force & récursif)

2.2 Création d'Arborescences Complexes en Une Ligne

L'une des forces majeures du Shell Bash est l'**extension d'accolades** (*Brace Expansion*), combinée avec l'option `-p` de la commande `mkdir`.

COMMANDE

```
# Créer une arborescence complète de projet web en 1 seule ligne :
mkdir -p mon_projet/{src/{css,js,images},docs,tests/unitaires}
```

RÉSULTAT GÉNÉRÉ INSTANTANÉMENT

```
mon_projet/
├── docs/
├── src/
│   ├── css/
│   ├── images/
│   └── js/
└── tests/
    └── unitaires/
```

Attention : Suppression Définitive en CLI

Sous Linux en ligne de commande, il n'y a **pas de corbeille**. La commande `rm -rf` supprime les fichiers et dossiers immédiatement et irréversiblement. Manipulez-la toujours avec précaution !

PARTIE 3

Caractères Spéciaux & Commandes de Filtre

3.1 Caractères Spéciaux (Wildcards / Jokers)

Les **Wildcards** permettent d'effectuer des recherches intelligentes et des manipulations par lots sur plusieurs fichiers en ciblant des motifs spécifiques.

Symbole	Signification	Exemple concret
`*`	Remplace 0, 1 ou plusieurs caractères quelconques.	`ls *.pdf` `rm test*`
`?`	Remplace un seul caractère exact.	`ls image?.jpg` (trouve *image1.jpg*, pas *image10.jpg*)
`[...]`	Définit une plage ou liste de caractères autorisés.	`ls doc[1-5].txt` `ls [a-z]*`
`{...}`	Expansion d'ensemble de chaînes précises.	`touch file_{A,B,C}.log`

EXEMPLES COMBINÉS D'UTILISATION DES WILDCARDS

```
# Copie tous les logs
cp /var/log/*.log ./sauvegarde/

# Supprime les rapports de 2020 à 2029
rm -f projet_202[0-9].pdf

# Cherche Admin... ou admin...
ls -l [aA]dmin*.txt
```

> **Astuce Pédagogique : Différence entre `*` et `?`**
>
> `*` est **gourmand** et remplace n'importe quelle longueur de texte. `?` est **strict** et exige la présence d'exactement un caractère à sa position.

3.2 Commandes de Filtre et de Recherche Avancée

Linux propose trois outils fondamentaux pour explorer et filtrer des volumes importants de fichiers et de données : `ls`, `grep` et `find`.

Outil	Rôle & Usage principal	Exemple type
`ls` (filtres)	Lister le contenu filtré d'un répertoire via des jokers.	`ls -lh /var/log/*.log`
`grep`	Recherche du **texte à l'intérieur** des fichiers.	`grep -i "error" /var/log/syslog`
`find`	Recherche des **fichiers** dans l'arborescence selon des critères.	`find /home -name "*.txt" -type f`

Combiner les Filtres avec le Pipe (|)

Le tube (*pipe* `|`) permet de rediriger la sortie d'une commande pour qu'elle devienne l'entrée de la suivante. C'est le cœur de la philosophie Unix.

EXEMPLES

```
# 1. Lister tous les fichiers et ne garder que ceux contenant "2026" :
ls -l /sauvegardes | grep "2026"

# 2. Chercher "FAILED" sans tenir compte de la casse (-i) dans les logs :
grep -i "failed" /var/log/*.log

# 3. Trouver tous les fichiers de plus de 50 Mo dans le dossier /var :
find /var -type f -size +50M -exec ls -lh {} \;
```

À retenir

- `find` cherche des **fichiers / dossiers** (par nom, type, taille...).
- `grep` cherche du **contenu textuel** à l'intérieur des fichiers.
- Le pipe `|` enchaîne les commandes : la sortie de l'une alimente l'entrée de l'autre.

PARTIE 4

Atelier Pratique Guidé pour les Apprenants

4.1 TP Pratique : Déploiement et Manipulation

Réalisez la séquence de commandes suivante dans votre terminal pour valider l'ensemble des concepts du cours :

```
# Étape 1 : Créer l'arborescence du projet
mkdir -p entreprise/{direction,technique/{dev,infra},hr}

# Étape 2 : Créer un jeu de fichiers de test
touch entreprise/technique/dev/{app_v1.py,app_v2.py,script.sh,notes.txt}
touch entreprise/direction/{rapport_2025.pdf,rapport_2026.pdf,budget.xlsx}

# Étape 3 : Copie ciblée avec Wildcard
cp entreprise/technique/dev/*.py entreprise/technique/infra/

# Étape 4 : Recherche avec find et grep
find entreprise -name "*.py"
grep -r "app" entreprise/
```

Synthèse de l'Apprenant - À Retenir

- Une commande se compose de : **Commande + Options (-flag) + Arguments**.
- `mkdir -p` et l'extension `{a,b}` permettent de générer des arborescences complexes instantanément.
- Les Wildcards (`*` , `?` , `[...]`) simplifient le ciblage de fichiers multiples.
- `find` cherche des fichiers/dossiers, tandis que `grep` cherche du contenu textuel dans les fichiers.